



QEMU

Проектирование цифровых вычислительных систем
Прутьянов В. В.



Эмуляция и симуляция

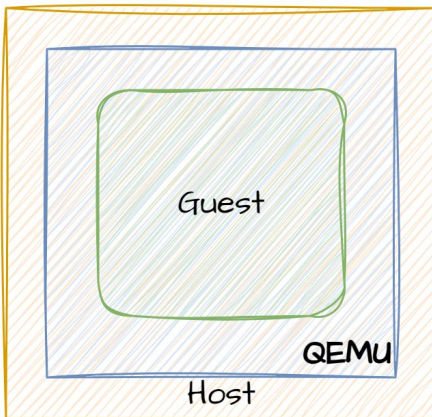
- RTL-симуляция
- Потактовая симуляция
- Функциональная симуляция
- **Эмуляция**

QEMU

- QEMU – инструмент для **системной эмуляции** (full-system emulation) и виртуализации с открытым исходным кодом (<https://gitlab.com/qemu-project/qemu>)

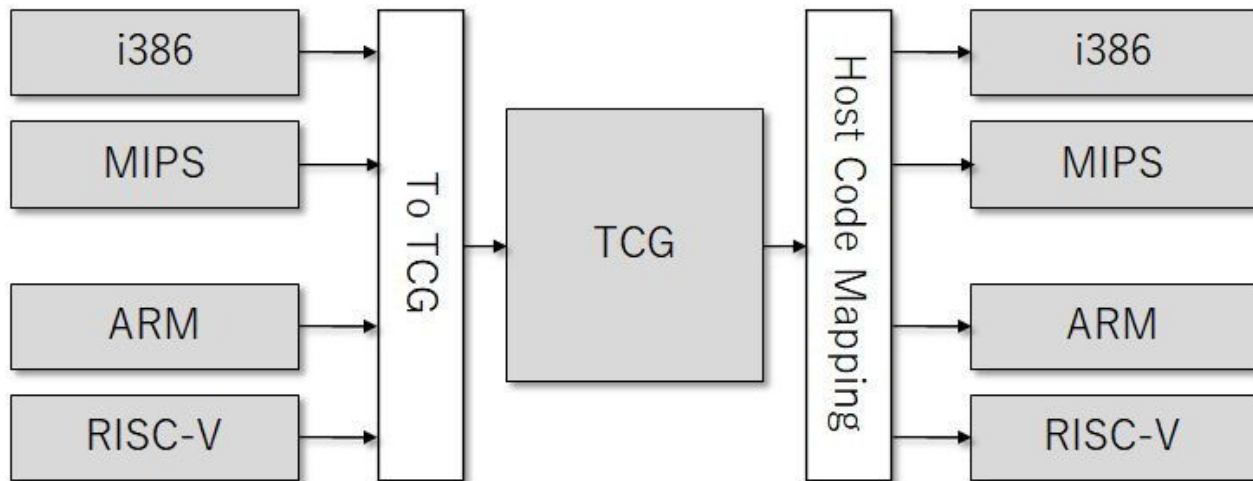
- Поддерживаемые гостевые архитектуры:

- x86
- **ARM**
- RISC-V
- MIPS
- LoongArch
- PowerPC
- SPARC
- s390x
- MicroBlaze
- etc.



- Поддержка Just-in-Time (JIT) трансляции гостевого кода для ускорения исполнения
- Поддержка эмуляции периферийных устройств (дисковых, сетевых, криптографических и т.д.)

Tiny Code Generator (TCG)



- TCG транслирует исполняемый код гостевой системы в код хостовой системы через собственное *промежуточное представление (IR)*

QEMU: Machine type и CPU model

- Machine type – эмулируемая аппаратная платформа, содержащая по умолчанию некоторый набор периферийных устройств
- Machine type не обязательно отражает реальную платформу
- CPU model – модель гостевого CPU, определяющая набор инструкций (ISA) с расширениями
- CPU model не привязан жестко к machine type

QEMU:ARM

- `qemu-system-arm` – для запуска 32-битного ARM
- `qemu-system-aarch64` – для запуска 32-битного и 64-битного ARM
- Около 100 типов машин, например Raspberry Pi 2B, STM32VL Discovery и т.д.
- Около 40 моделей CPU, например Cortex-A15, Cortex-M3 и т.д.
- Если не стоит цель эмуляции реальной аппаратной платформы, есть тип машины `virt`, позволяющий изменять параметры в широких пределах (CPU по умолчанию – Cortex-A15)

Вывести список типов машин:

```
qemu-system-arm -M ?
```

Вывести список моделей CPU для машины `virt`:

```
qemu-system-arm -M virt -cpu ?
```

Запустить машину `virt` с Cortex-A7 и 128MiB памяти без графики:

```
qemu-system-arm -M virt -cpu cortex-a7 -m 128M -nographic
```

QEMU: Monitor

QEMU monitor (HMP) позволяет передавать команды запущенной машине, например:

- `stop` – приостановить работу машины
- `cont` – продолжить работу машины
- `info status` – запросить состояние машины
- `X` – вывести данные из памяти по виртуальному адресу
- `xp` – вывести данные из памяти по физическому адресу
- `info registers` – вывести текущее содержимое CPU регистров
- `info mtree` – вывести дерево регионов памяти
- etc.

Запуск машины с перенаправлением монитора в консоль

```
qemu-system-arm -M virt -monitor stdio
```

Запуск машины с перенаправлением монитора в порт 1234 по протоколу Telnet

```
qemu-system-arm -M virt -monitor telnet:127.0.0.1:1234,server,nowait  
telnet 127.0.0.1 1234
```

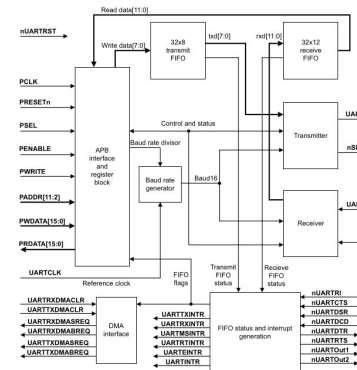
qemu-system-arm: virt

Платформа virt [hw/arm/virt.c] содержит следующую периферию:

- ARM Cortex-A15 (ARMv7, 32-bit) (по умолчанию)
- ARM PL011 UART [hw/char/pl011.c]
- ARM PL031 Real-Time Clock (RTC) [hw/rtc/pl031.c]
- ARM PL061 GPIO controller [hw/gpio/pl061.c]
- ARM Global Interrupt Controller (GIC), версия зависит от CPU
- etc.

Пример: вывод строки в PL011 UART

- ARM PrimeCell UART (PL011)
 - В реальных ARM SoC это UART RX/TX на APB шине
 - В QEMU это устройство, способное выводить символы в терминал
 - В машине **virt** MMIO-регистры PL011 отображены в 0x900'0000+0xFFF
 - Адрес можно узнать из **hw/arm/virt.c**
 - Адрес можно узнать из **info mtree** в мониторе
 - Root memory region: system
 - 0000000000000000
 - 090000000-09000fff (prio 0, i/o): pl011
 - Адрес можно узнать из Device Tree
- | Bits | Name | Func |
|------|------|-----------------|
| 7:0 | DATA | Receiv
Trans |



Bits	Name	Function
7:0	DATA	Receive (read) data character. Transmit (write) data character.

Offset	Name	Type	Reset	Width	Description
0x000	UARTDR	RW	0x---	12/8	<i>Data Register, UARTDR on page 3-5</i>

Пример: вывод строки в PL011

```
#define UARTDR 0x90000000

void pl011_puts(const char *s) {
    while (*s) {
        *(volatile unsigned char *)UARTDR = *s;
        s++;
    }
}

void entry() {
    pl011_puts("Hello world!\n");
}
```

hello.c

```
.global _start
_start:
    ldr sp, =stack_top // Настройка стека
    bl entry
    b .
```

start.s

```
$ qemu-system-arm -M virt -nographic -kernel hello.bin
Hello world!
```

```
all:
$(AS) start.s -o start.o
$(CC) hello.c -g -c -o hello.o
$(LD) -T virt.ld hello.o start.o -o hello.elf
$(OBJCOPY) -O binary hello.elf hello.bin
```

Makefile

```
ENTRY(_start)
SECTIONS
{
    . = 0x40010000; /* Точка входа */
    .start . : { start.o(.text) }
    .text : { *(.text) }
    .data : { *(.data) }
    .bss : { *(.bss COMMON) }
    . = ALIGN(8);
    . = . + 0x1000; /* 4KiB стека */
    stack_top = .
}
```

virt.ld

Пример: inc-dev

- Создадим собственное устройство в QEMU под названием `inc-dev`
- Устройство имеет один 64-битный регистр, отображенный в память по адресу `0x900'8000`
- Создадим *тип экземпляров* устройства и зарегистрируем его в QEMU Object Model (QOM)
- Создадим регион памяти (`MemoryRegion`) и определим `read/write` обработчики
 - Запись: устанавливает значение счетчика
 - Чтение: возвращает старое значение счетчика и увеличивает его на 1
- Подключим устройство (`SysBusDevice`) к системной шине в ARM-машине virt

QEMU Object Model (QOM)

- QOM реализует ООП средствами языка C
- Объекты – устройства, шины, регионы памяти и т.д.
- Структуры экземпляров (instance) описывают данные конкретного экземпляра объекта
- Структуры классов (class) описывают данные класса объектов и виртуальные методы
- Возможно динамическое создание классов и объектов в рантайме и интроспекция
- Приведение типов происходит через макросы
- Иерархию объектов можно посмотреть через `info qom-tree` на запущенной машине

QOM: создание нового устройства

1. Создать структуру типа `TypeInfo`: структура описывает новый тип для QOM, включая его имя, родительский тип и размер структуры экземпляра
2. Реализовать функции инициализации:
 - Функция `instance_init` задает начальное состояние для каждого нового экземпляра
 - Функция `class_init` задает свойства всего класса и указатели на виртуальные методы
3. Объявить структуру экземпляра: структура, которая содержит поле `parent_obj` от родительского типа, хранит состояние конкретного устройства (например, значения регистров, регионы памяти и т.д.)
4. Зарегистрировать тип: структура `TypeInfo` регистрируется (часто статически), делая новый тип доступным для создания внутри QEMU

Memory Region

MemoryRegion представляет непрерывный регион адресного пространства гостевой системы для:

- RAM (`memory_region_init_ram_*`)
- ROM (`memory_region_init_rom_*`)
- MMIO (`memory_region_init_io`)
- Псевдоним (`alias`) для другого региона (`memory_region_init_alias`)

MemoryRegion для MMIO-регионов используют **MemoryRegionOps**, определяющий:

- Обработчик чтения (`read`)
- Обработчик записи (`write`)
- Порядок байтов (`endianness`)
- Ограничения на размер доступа (`valid.min_access_size`, `valid.max_access_size`)
- Обработчик проверки корректности доступа (`valid.accepts`)

SysBus

- SysBus – абстракция основной системной шины в виртуальной машине QEMU
- Большинство устройств, работающих через MMIO, подключаются через SysBus
- `SysBusDevice` – тип устройства на SysBus шине, от которого можно унаследовать тип своего устройства
- `sysbus_init_mmio(SysBusDevice *dev, MemoryRegion *memory)` – добавляет MMIO-регион на шину
- `sysbus_mmio_map(SysBusDevice *dev, int n, hwaddr addr)` – отображает устройство на адрес `addr`
- `sysbus_create_simple(const char *name, hwaddr addr, qemu_irq irq)` – создает устройство типа `name` с отображением на адрес `addr` и подключает прерывание `irq` (если не `NULL`)

include/hw/misc/inc-dev.h

```
#include "hw/sysbus.h"
#include "exec/memory.h"
#include "qom/object.h"

#include "qemu/osdep.h"
#include "qemu/log.h"

#define INC_DEV_SIZE 0x1000

// Структура экземпляра устройства inc-dev
struct IncDev {
    SysBusDevice parent_obj; // Данные SysBusDevice (для наследования)
    MemoryRegion iomem;      // Поле объекта: регион памяти
    uint64_t cnt;            // Поле объекта: счетчик
};

#define TYPE_INC_DEV "inc-dev"
OBJECT_DECLARE_SIMPLE_TYPE(IncDev, INC_DEV) // Определяет функцию INC_DEV для приведения к типу
```

hw/misc/inc-dev.c

```
#include "hw/misc/inc-dev.h"

static uint64_t inc_dev_read(void *opaque, hwaddr addr, unsigned len) {
    IncDev *dev = opaque;
    return dev->cnt++;
}

static void inc_dev_write(void *opaque, hwaddr addr, uint64_t val, unsigned len) {
    IncDev *dev = opaque;
    dev->cnt = val;
}

static const MemoryRegionOps inc_dev_ops = {
    .read = inc_dev_read,
    .write = inc_dev_write,
    .endianness = DEVICE_LITTLE_ENDIAN,
};
```

hw/misc/inc-dev.c

```
static void inc_dev_init(Object *obj) {
    IncDev *s = INC_DEV(obj);
    SysBusDevice *sbd = SYS_BUS_DEVICE(obj);
    qemu_log("%s: enter\n", __func__);
    memory_region_init_io(&s->iomem, OBJECT(s), &inc_dev_ops, s, TYPE_INC_DEV, INC_DEV_SIZE);
    sysbus_init_mmio(sbd, &s->iomem);
}

static void inc_dev_class_init(ObjectClass *klass, void *data) {
    DeviceClass *dc = DEVICE_CLASS(klass);
    dc->desc = TYPE_INC_DEV;
    set_bit(DEVICE_CATEGORY_MISC, dc->categories);
}

static const TypeInfo inc_dev_info = {
    .name          = TYPE_INC_DEV,
    .parent        = TYPE_SYS_BUS_DEVICE,
    .instance_size = sizeof(IncDev),
    .instance_init = inc_dev_init,
    .class_init    = inc_dev_class_init,
};

static void inc_dev_register_types(void) {
    type_register_static(&inc_dev_info);
}

type_init(inc_dev_register_types) // Регистрация типа inc-dev
```

include/hw/arm/virt.h

```
enum {  
    VIRT_FLASH,  
    VIRT_MEM,  
    VIRT_CPUPERIPHS,  
    VIRT_GIC_DIST,  
    VIRT_GIC_CPU,  
    .....  
    VIRT_SMMU,  
    VIRT_UART0,  
    VIRT_INC_DEV,  
    VIRT_MMIO,  
    VIRT_RTC,  
    VIRT_FW_CFG,  
    VIRT_PCIE,  
    VIRT_PCIE_MMIO,  
    VIRT_PCIE_PIO,  
    VIRT_PCIE_ECAM,  
    VIRT_PLATFORM_BUS,  
    VIRT_GPIO,  
    .....  
    VIRT_LOWMEMMAP_LAST,  
};
```

hw/arm/virt.c

```
/* Addresses and sizes of our components.
 * 0..128MB is space for a flash device so we can run bootrom code such as UEFI.
 * 128MB..256MB is used for miscellaneous device I/O.
 .....
 */
static const MemMapEntry base_memmap[] = {
    /* Space up to 0x8000000 is reserved for a boot ROM */
    [VIRT_FLASH] = { 0, 0x08000000 },
    [VIRT_CPUPERIPHS] = { 0x08000000, 0x00020000 },
    /* GIC distributor and CPU interfaces sit inside the CPU peripheral space */
    [VIRT_GIC_DIST] = { 0x08000000, 0x00010000 },
    [VIRT_GIC_CPU] = { 0x08010000, 0x00010000 },
    [VIRT_GIC_V2M] = { 0x08020000, 0x00001000 },
    .....
    [VIRT_UART0] = { 0x09000000, 0x00001000 },
    [VIRT_INC_DEV] = { 0x09008000, 0x00001000 },
    [VIRT_RTC] = { 0x09010000, 0x00001000 },
    .....
}
```

hw/arm/virt.c

```
static void create_inc_dev(VirtMachineState *vms)
{
    hwaddr base = vms->memmap[VIRT_INC_DEV].base;

    // Создает объект устройства inc-dev и добавляет его на системную шину
    sysbus_create_simple("inc-dev", base, NULL /* No IRQs */);
}

static void machvirt_init(MachineState *machine)
{
    VirtMachineState *vms = VIRT_MACHINE(machine);
    MemoryRegion *systemem = get_system_memory();

    .....
    create_uart(vms, VIRT_UART0, systemem, serial_hd(0), false);
    .....
    create_inc_dev(vms);
    create_rtc(vms);
    .....
}
```

hw/misc/

```
system_ss.add(when: 'CONFIG_INC_DEVICE', if_true: files('inc-dev.c'))
```

meson.build

```
config INC_DEVICE
    bool
    default y
```

Kconfig

Monitor

(qemu) **info mtree**

memory-region: system

0000000000000000-ffffffffffffffff (prio 0, i/o): system

.....

0000000009000000-0000000009000fff (prio 0, i/o): pl011

0000000009008000-0000000009008fff (prio 0, i/o): inc-dev

0000000009010000-0000000009010fff (prio 0, i/o): pl031

(qemu) **info qtree**

bus: main-system-bus

type System

.....

dev: inc-dev, id ""

mmio 0000000009008000/00000000000001000

.....

(qemu) **info qom-tree**

/machine (virt-9.2-machine)

/unattached (container)

.....

/device[4] (inc-dev)

/inc-dev[0] (memory-region)

.....

Тест: hello.c

```
#define UARTDR 0x90000000
#define INCDEV 0x90080000

typedef unsigned char uint8_t;
typedef long unsigned int uint64_t;

void pl011_putc(const uint8_t c) {
    *(volatile uint8_t *)UARTDR = c;
}

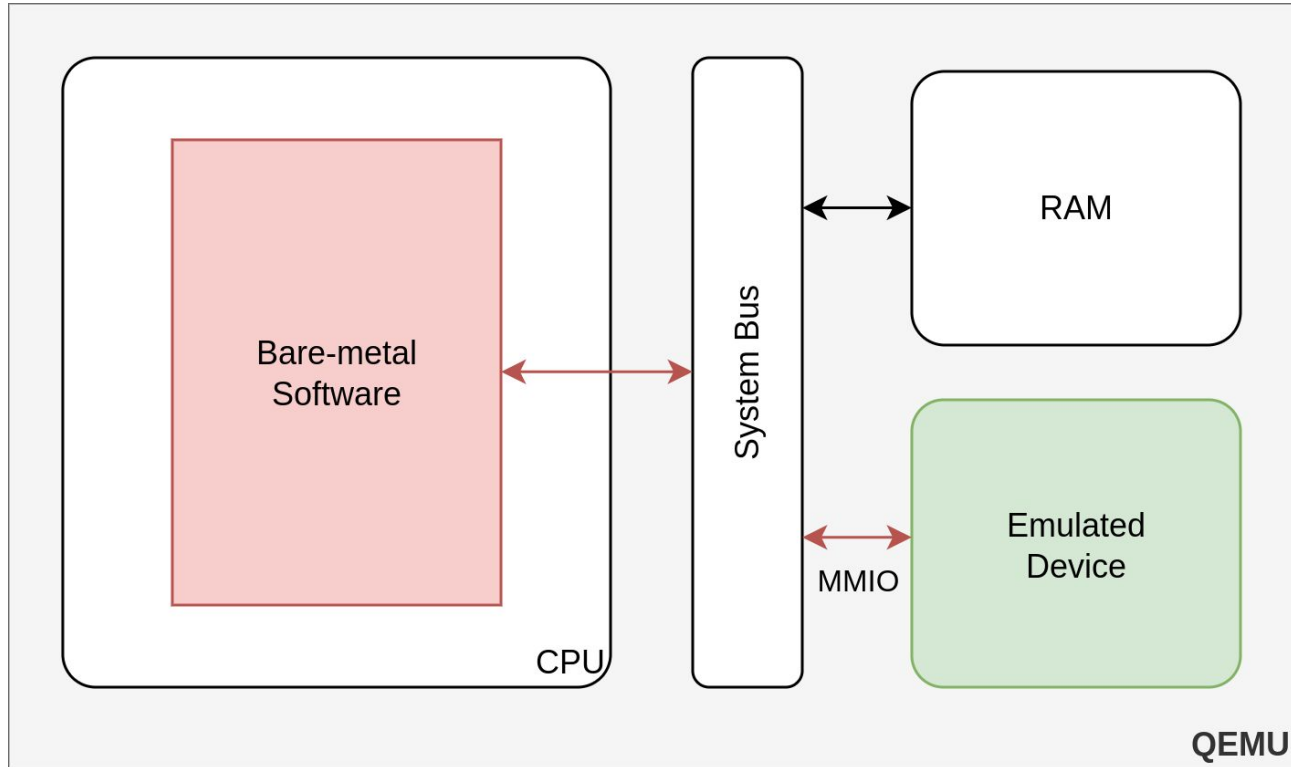
void incdev_set(const uint64_t n) {
    *(volatile uint64_t *)INCDEV = n;
}

uint64_t incdev_get() {
    return *(volatile uint64_t *)INCDEV;
}
```

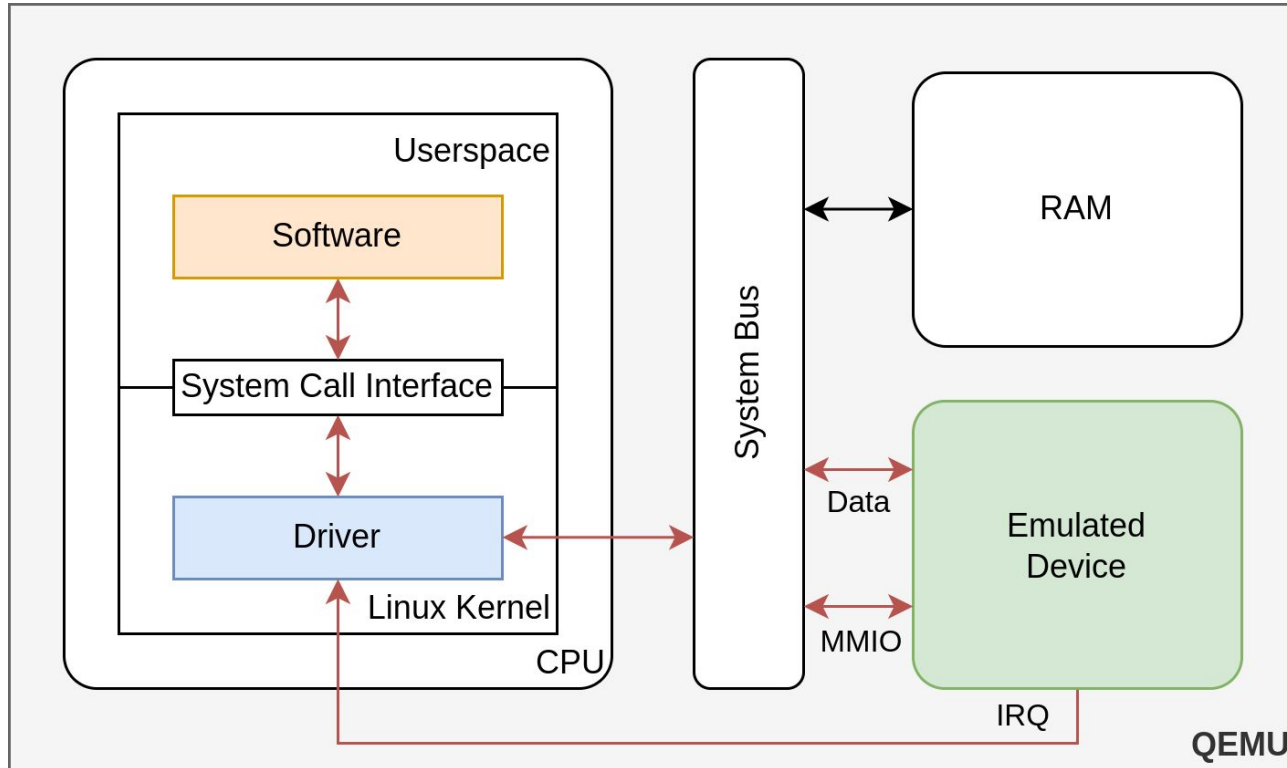
```
void entry() {
    incdev_set('A');
    for (int i = 0; i < 8; i++)
        pl011_putc(incdev_get());
    pl011_putc('\n');
}
```

```
$ qemu-system-arm -M virt -nographic -kernel hello.bin
inc_dev_init: enter
ABCDEFGH
```

QEMU + Bare-metal SW



QEMU + Linux + User-space SW



Ссылки

- [QEMU's documentation](#)
- [Airbus Security Lab: QEMU Internals](#)

Задание

1. Сделать для QEMU модель устройства mul-dev с 3 регистрами, доступными через MMIO
2. Запись в 2 регистра сохраняет множители, чтение из 3-го регистра возвращает результат умножения
3. Добавить устройство в ARM-машину virt
4. Проверить работу устройства с помощью bare-metal программы на языке C